

STACK TECNICO E ARCHITETTURA

Sistema web per raccolta, consolidamento e consultazione di annunci

Documento tecnico basato sul brief di progetto del 24 agosto 2026

Versione proposta per sviluppo e preventivazione

1. Obiettivo del sistema

Il progetto prevede un applicativo web riservato capace di acquisire annunci pubblicati su un insieme configurabile di siti, normalizzarli e renderli consultabili tramite un archivio unificato. Il numero telefonico normalizzato costituisce la chiave funzionale di raggruppamento, mentre ogni singola occorrenza mantiene provenienza, URL, date, contenuti e media.

- Centralizzare annunci provenienti da più portali.
- Cercare rapidamente una scheda tramite numero di telefono.
- Deduplicare senza perdere la provenienza delle singole occorrenze.
- Applicare una regola deterministica per la selezione dell'annuncio canonico.
- Gestire media originali e copie derivate in modo tracciabile.
- Classificare i media distinguendo contenuti espliciti e non espliciti.
- Generare un riepilogo della scheda distinguendo dati degli annunci e contributi del forum.
- Esportare dati e media tramite modalità separate e controllate.

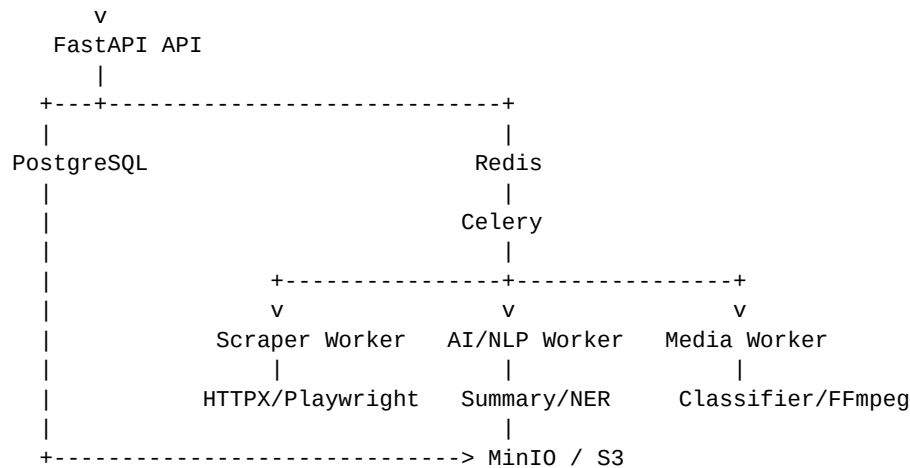
2. Stack tecnico consigliato

Area	Tecnologie
Frontend	React, Vite, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query
Backend API	Python 3.13, FastAPI, Pydantic
ORM / Migrazioni	SQLAlchemy 2, Alembic
Database	PostgreSQL 17
Cache / Broker	Redis
Job asincroni	Celery + Celery Beat
Scraping HTTP	httpx, lxml / BeautifulSoup
Scraping browser	Playwright
Storage media	MinIO oppure storage S3-compatible
Elaborazione immagini	Pillow, imagehash / pHash
Video	FFmpeg
Classificazione media	ONNX Runtime / Transformers, modello dedicato
AI / NLP	LLM API oppure modello locale, output strutturato
Autenticazione	Keycloak oppure OAuth2/JWT gestito dal backend
Observability	Sentry, Prometheus, Grafana, Loki
Reverse proxy	Nginx oppure Apache2
Container	Docker, Docker Compose
CI/CD	GitHub Actions

3. Architettura logica

La soluzione viene organizzata come un'applicazione modulare con API centrale, worker asincroni e connettori indipendenti per le diverse fonti. Il frontend non comunica direttamente con gli scraper: tutte le operazioni passano dal backend.

React Frontend
|



4. Frontend

Per il pannello amministrativo è consigliato React con Vite e TypeScript. Tailwind CSS e shadcn/ui consentono di costruire rapidamente una UI coerente e responsive, mentre TanStack Query gestisce cache, retry e sincronizzazione con le API.

4.1 Schermate principali

```

/login
/dashboard
/search
/records/:id
/sources
/exports
/admin
  
```

- Ricerca esatta per numero telefonico.
- Filtri per fonte, stato e data.
- Scheda unificata con annuncio canonico.
- Elenco deduplicato di tutte le occorrenze.
- Cronologia delle acquisizioni e modifiche.
- Galleria media con classificazione.
- Comandi separati per le modalità di esportazione.
- Sezione amministrativa per fonti, priorità, utenti e classificatori.

5. Backend API

FastAPI è indicato perché il progetto è fortemente orientato a scraping, data processing e AI. Mantenendo crawler, normalizzazione, NLP e API nello stesso ecosistema Python si riduce la complessità operativa e si facilita il riuso del codice.

- REST API versionata, ad esempio /api/v1.
- Validazione input/output con Pydantic.
- SQLAlchemy 2 per l'accesso dati.

- Alembic per migrazioni database.
- OpenAPI automatica per documentazione delle API.
- RBAC per separare amministratori, operatori e utenti in sola consultazione.

6. Scraping e connettori

Ogni fonte deve essere implementata come connettore separato e manutenibile. L'interfaccia comune consente di sostituire o aggiornare un singolo scraper senza influenzare gli altri.

```
scrapers/
|-- base.py
|-- escort_advisor.py
|-- bakeca_incontri.py
|-- moscarossa.py
|-- megaescort.py
|-- escortforumit.py
|-- escortacom.py
|-- rosa_rossa.py
|-- torino_erotica.py
`-- punterforum.py

class Scraper:
    async def discover(self):
        ...

    async def scrape_ad(self, url):
        ...

    async def download_media(self, ad):
        ...

    async def normalize(self, data):
        ...
```

Per pagine HTML tradizionali si preferiscono httpx e lxml/BeautifulSoup. Playwright viene utilizzato solo quando il contenuto richiede rendering JavaScript o interazione browser. Ogni connettore deve rispettare le pagine consentite, i limiti di frequenza, le condizioni d'uso e le indicazioni tecniche applicabili.

7. Orchestrazione asincrona

Le attività di raccolta, elaborazione media, deduplicazione e generazione dei riepiloghi non devono bloccare le richieste HTTP dell'utente. Redis viene usato come broker/cache e Celery per distribuire i job.

- Celery Beat per schedulare scansioni periodiche.
- Code separate per scraping, media e AI.
- Retry con backoff per errori temporanei.
- Dead-letter / stato di errore per job non recuperabili.
- Rate limit per singola fonte.
- Metriche su durata, successi, errori e numero di elementi acquisiti.

8. Modello dati PostgreSQL

Il numero telefonico non viene utilizzato come primary key. È preferibile un identificativo tecnico interno, mantenendo il numero cifrato e un indice crittografico per la ricerca esatta.

```
record
-----
id UUID
phone_encrypted
phone_lookup_hash
canonical_ad_id
created_at
updated_at

advertisement
-----
id UUID
record_id UUID
source_id
source_url
title
description
content_hash
first_seen_at
last_seen_at
scraped_at
confidence
status

media
-----
id UUID
advertisement_id UUID
original_object_key
derived_object_key
sha256
perceptual_hash
mime_type
classification
classification_confidence
classifier_version
created_at
```

8.1 Tabelle aggiuntive

- sources: configurazione, priorità e stato delle fonti.
- canonical_history: storico della selezione canonica e override amministrativi.
- scrape_runs: stato e metriche delle scansioni.
- scrape_errors: errori strutturati per fonte e URL.
- media_classification_history: risultati e override manuali.
- summary_versions: versioni del riepilogo AI e fonti utilizzate.
- export_jobs: richieste di esportazione, stato, manifesto e audit.
- audit_log: operazioni rilevanti compiute dagli utenti.

9. Protezione del numero telefonico

Per supportare la ricerca esatta senza indicizzare il numero in chiaro, si può utilizzare una coppia cifratura + indice deterministico.

```
Input:
+39 333 123 4567

Normalizzazione:
+393331234567

Valore conservato:
phone_encrypted = AES-256-GCM(...)
phone_lookup_hash = HMAC-SHA256(secret_key, normalized_phone)
```

Il database effettua la ricerca tramite `phone_lookup_hash`; il numero viene decifrato soltanto quando necessario e in un contesto autorizzato. Le chiavi devono essere esterne al database e gestite tramite segreti applicativi.

10. Deduplicazione

La deduplicazione deve essere multilivello. Nessun singolo segnale è sufficiente per gestire copie modificate, URL diversi o media ricompresi.

1. Numero telefonico normalizzato
2. URL normalizzato
3. SHA-256 del contenuto
4. Confronto dei campi principali
5. Similarità testuale, se necessaria
6. pHash / hash percettivo delle immagini

Le occorrenze considerate identiche o sostanzialmente equivalenti possono essere presentate come un unico elemento logico, conservando comunque tutti i riferimenti alle fonti e agli URL originali.

11. Selezione dell'annuncio canonico

La selezione canonica deve essere deterministica e implementata in un servizio dedicato, non affidata a un modello generativo.

```
if valid_bakeca_ads:
    canonical = most_recent_valid_bakeca_ad
else:
    canonical = most_recent_valid_ad

tie_breaker:
1. maggior numero di campi validi
2. priorità configurabile della fonte
```

Un amministratore può forzare manualmente la fonte canonica. Ogni modifica registra valore precedente, nuovo valore, motivazione, utente e timestamp.

12. Storage e gestione media

Immagini e video non devono essere salvati direttamente nel database. Il database conserva riferimenti e metadati, mentre gli oggetti vengono archiviati in MinIO o in uno storage S3-compatible.

```
media/{record_uuid}/{media_uuid}/original.jpg
media/{record_uuid}/{media_uuid}/derived.jpg
```

- L'originale non viene sovrascritto.
- Ogni trasformazione genera una copia derivata.
- SHA-256 per integrità e deduplicazione esatta.
- pHash per individuare immagini visivamente equivalenti.
- Metadati con fonte, URL, data acquisizione e classificazione.
- FFmpeg per video e generazione di preview, quando necessario.

Le operazioni di rimozione di filigrane vanno abilitate soltanto nei casi autorizzati descritti nel brief, mantenendo sempre l'originale e la tracciabilità dell'operazione.

13. Classificazione dei media

La classificazione dei contenuti viene eseguita in un worker separato. Per la produzione è utile esportare il modello in ONNX e utilizzare ONNX Runtime, riducendo consumo di memoria e tempi di avvio.

```
{
  "classification": "explicit",
  "confidence": 0.97,
  "model": "classifier-name",
  "version": "1.4.2",
  "manual_override": false
}
```

- Versione del classificatore sempre registrata.
- Confidence score conservato nel database.
- Possibilità di revisione e override manuale.
- Storico dei cambi di classificazione.
- Modalità export che include esclusivamente media classificati come non espliciti.

14. Riepilogo AI / NLP

Per ogni numero può essere generato un riepilogo utilizzando le descrizioni degli annunci associati e i contenuti pertinenti della fonte forum prevista dal brief. Il modello deve produrre output strutturato e mantenere la distinzione tra informazioni provenienti dagli annunci, contenuti del forum e affermazioni non verificate.

```
{
  "summary": "...",
  "advertisement_information": [],
  "forum_information": [],
  "unverified_claims": [],
  "sources": []
}
```

}

Il riepilogo deve essere versionato e deve conservare le fonti utilizzate. È preferibile rigenerarlo soltanto quando cambia il set di occorrenze rilevanti.

15. Esportazioni

Si prevedono tre flussi distinti:

- Esportazione dati testuali.
- Esportazione annuncio completo con tutti i media consentiti.
- Esportazione annuncio completo con soli media classificati come non espliciti.

Ogni pacchetto media deve contenere un manifesto con nome file, tipo, impronta di deduplicazione, URL di origine, data di acquisizione e classificazione.

16. Autenticazione e autorizzazioni

L'applicazione è riservata. Se è già disponibile un'infrastruttura IAM, Keycloak è una buona scelta; in alternativa FastAPI può gestire OAuth2/JWT internamente.

- Ruoli suggeriti: Admin, Operator, Viewer.
- MFA consigliata per gli account amministrativi.
- Token a durata limitata.
- Audit delle esportazioni e degli override.
- Principio del minimo privilegio.
- Secret e chiavi crittografiche mai inseriti nel repository.

17. Logging, metriche e audit

- Sentry per eccezioni applicative.
- Prometheus per metriche di API e worker.
- Grafana per dashboard operative.
- Loki per aggregazione centralizzata dei log.
- structlog o logging JSON nel backend Python.
- Audit log separato per azioni amministrative e operazioni sensibili.

18. Docker e deployment

Per la prima versione Docker Compose è sufficiente. Kubernetes non è necessario finché il carico o i requisiti di alta disponibilità non lo giustificano.

```
services:
  frontend
  api
  worker-scraper
  worker-media
```

```
worker-ai
scheduler
redis
postgres
minio
prometheus
grafana
loki
```

Il reverse proxy espone esclusivamente frontend e API. PostgreSQL, Redis e MinIO devono rimanere su rete interna o essere protetti da firewall e autenticazione.

19. CI/CD

- GitHub Actions su pull request e branch principale.
- Lint frontend e backend.
- Test unitari e di integrazione.
- Build frontend.
- Build immagini Docker.
- Migrazioni Alembic gestite nel processo di deploy.
- Container scanning e dependency scanning consigliati.
- Deploy staging prima della produzione.

20. Struttura repository suggerita

```
project/
|-- frontend/
|   |-- src/
|   |-- ...
|-- backend/
|   |-- app/
|   |   |-- api/
|   |   |-- models/
|   |   |-- schemas/
|   |   |-- services/
|   |   |-- scrapers/
|   |   |-- workers/
|   |   |-- security/
|   |   |-- main.py
|   |-- migrations/
|   |-- tests/
|-- infra/
|   |-- nginx/
|   |-- prometheus/
|   |-- grafana/
|-- docker-compose.yml
|-- .env.example
`-- README.md
```

21. Sicurezza e conformità operativa

Il sistema tratta dati provenienti da fonti pubblicate online e associa contenuti a numeri telefonici. La progettazione deve quindi includere minimizzazione dei dati, controllo degli accessi, cifratura, audit, retention configurabile e procedure per correzione/cancellazione quando applicabili. I crawler devono rispettare condizioni d'uso, limiti di frequenza e indicazioni tecniche delle fonti. Le funzionalità sui media devono mantenere gli originali e applicare trasformazioni soltanto quando esiste il presupposto previsto dal brief. La validazione legale/GDPR definitiva richiede comunque una verifica specialistica del caso d'uso.

22. Stack finale raccomandato

FRONTEND

React + Vite + TypeScript
Tailwind CSS + shadcn/ui
TanStack Query

BACKEND

Python 3.13
FastAPI + Pydantic
SQLAlchemy 2 + Alembic

SCRAPING

httpx + lxml/BeautifulSoup
Playwright

ASYNC

Celery + Redis

DATABASE

PostgreSQL 17

MEDIA

MinIO / S3
Pillow + pHash
FFmpeg

AI

ONNX Runtime / Transformers
LLM API o modello locale

AUTH

Keycloak o OAuth2/JWT

OBSERVABILITY

Sentry + Prometheus + Grafana + Loki

INFRA

Docker + Docker Compose
Nginx/Apache
GitHub Actions

23. Conclusione

Il nucleo consigliato è React + FastAPI + PostgreSQL + Redis/Celery + Playwright + MinIO. Questa architettura mantiene separati API, scraping, AI e processing dei media senza introdurre subito la complessità di microservizi o Kubernetes. I connettori per fonte, le code Celery e lo storage a oggetti consentono di scalare progressivamente il sistema e di aggiungere nuove fonti senza cambiare l'impianto generale.

Documento tecnico - 24 agosto 2026